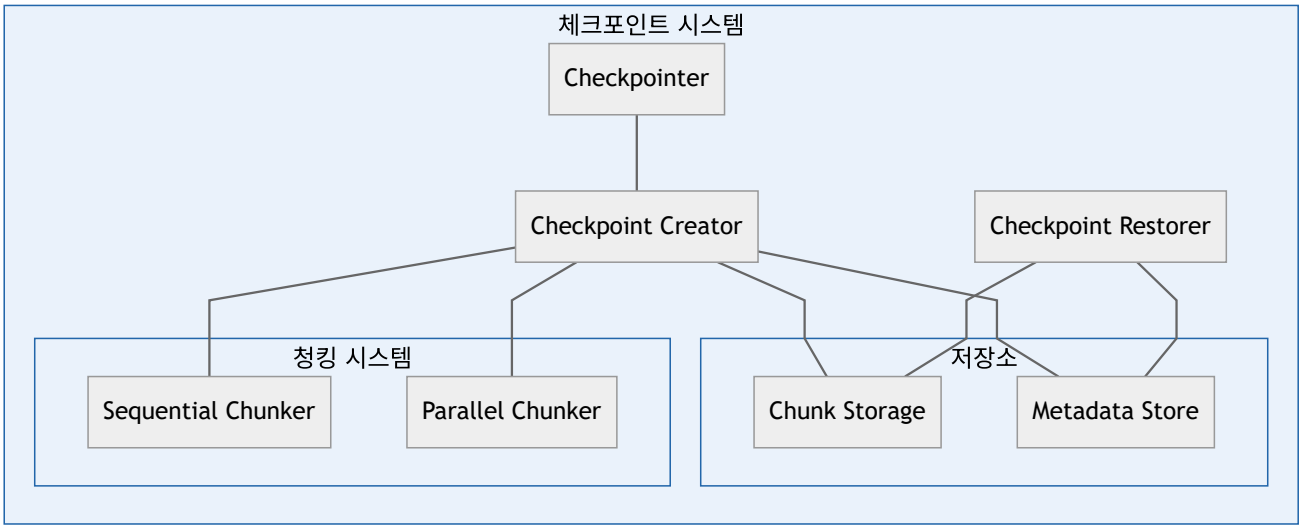
체크포인트 시스템

소개

체크포인트 시스템은 Oasis Core의 상태 동기화를 위한 핵심 메커니즘으로, 대용량 상태 데이터를 효율적으로 전송하고 복원할 수 있게 해줍니다. 체크포인트는 특정 시점의 MKVS 상태를 청크 단위로 분할하여 저장하고, 이를 통해 빠른 상태 동기화를 지원합니다.

아키텍처 개요

핵심 구성 요소



주요 특징

- **청크 기반 분할**: 대용량 상태를 작은 청크로 분할하여 처리
- **병렬 처리**: 다중 스레드를 활용한 병렬 청킹

- **검증 가능성**: Merkle 증명을 통한 청크 무결성 검증
- **점진적 복원**: 청크별 점진적 상태 복원

체크포인트 생성

1. 체크포인트 메타데이터

```
type Metadata struct {
Version  uint16  `json:"version"`  // 체크포인트 버전
Root node.Root `json:"root"`  // 루트 정보
Chunks []hash.Hash `json:"chunks"`  // 청크 해시 목록
}

type ChunkMetadata struct {
Version  uint16  `json:"version"`  // 체크포인트 버전
Root node.Root `json:"root"`  // 루트 정보
Index  uint64  `json:"index"`  // 청크 인덱스
Digest hash.Hash `json:"digest"`  // 청크 해시
}
```

2. 체크포인트 생성 과정

기본 생성 프로세스

```
func  createCheckpoint(creator Creator, root node.Root, chunkSize  uint64) (*Met
```

```
// 체크포인트 생성
metadata, err := creator.CreateCheckpoint(
context.Background(),
root,
chunkSize,
4, // 4개 스레드 사용
)
if err != nil {
return nil, err
}
fmt.Printf("Created checkpoint with %d chunks\n", len(metadata.Chunks))
return metadata, nil
}
```

순차 청킹 (Sequential Chunking)

```
type seqChunker struct {
ndb db.NodeDB
root node.Root
chunkSize uint64
}

func (sc *seqChunker) chunk(ctx context.Context, wf writerFactory) ([]hash.Hash,
```

```go
tree := mkvs.NewWithRoot(nil, sc.ndb, sc.root)
defer tree.Close()

var chunks []hash.Hash
var nextOffset node.Key
for {
// 청크 생성
idx, writer, err := wf.next()
if err != nil {
return nil, err
}
chunkHash, nextOffset, err := sc.createChunk(ctx, tree, nextOffset, writer)
writer.Close()
if err != nil {
return nil, err
}
chunks = append(chunks, chunkHash)
// 완료 확인
if nextOffset == nil {
break
}
}
return chunks, nil
```

```
}
```

병렬 청킹 (Parallel Chunking)

```
type parallelChunker struct {
ndb db.NodeDB
root node.Root
chunkSize  uint64
threads  uint16
}

func (pc *parallelChunker) chunk(ctx context.Context, wf writerFactory) ([]hash.
// 서브트리 생성
root, err := newSubtree(pc.ndb, pc.root)
if  err != nil {
return  nil, err
}
pending := []*subtree{root}
var  chunks []hash.Hash
for  len(pending) > 0 {
// 작업 분할
pending, err = pc.splitTasks(pending)
if  err != nil {
```

```go
return nil, err
}
// 병렬 청크 생성
hashes, err := pc.createChunks(ctx, wf, pending)
if err != nil {
return nil, err
}
chunks = append(chunks, hashes...)
pending = pc.filterFinished(pending)
}
return chunks, nil
}
```

3. 청크 압축 및 저장

```go
func writeChunk(proof *syncer.Proof, w io.Writer) (hash.Hash, error) {
hb := hash.NewBuilder()
sw := snappy.NewBufferedWriter(io.MultiWriter(w, hb))
enc := cbor.NewEncoder(sw)
// 증명 엔트리들을 압축하여 저장
for _, entry := range proof.Entries {
if err := enc.Encode(entry); err != nil {
return hash.Hash{}, err
```

```
}
}
if err := sw.Close(); err != nil {
return hash.Hash{}, err
}
return hb.Build(), nil
}
```

체크포인트 복원

1. 복원 프로세스

복원 시작

```
func startRestore(restorer Restorer, metadata *Metadata) error {
// 복원 시작
if err := restorer.StartRestore(context.Background(), metadata); err != nil {
return err
}
fmt.Printf("Started restore for checkpoint with %d chunks\n", len(metadata.Chunk
return nil
}
```

청크별 복원

```go
func restoreChunks(restorer Restorer, chunkProvider ChunkProvider, metadata *Me
ctx := context.Background()
for i, chunkHash := range metadata.Chunks {
// 청크 메타데이터 생성
chunkMeta := &ChunkMetadata{
Version: metadata.Version,
Root: metadata.Root,
Index: uint64(i),
Digest: chunkHash,
}
// 청크 데이터 가져오기
var buf bytes.Buffer
if err := chunkProvider.GetCheckpointChunk(ctx, chunkMeta, &buf); err != nil {
return err
}
// 청크 복원
done, err := restorer.RestoreChunk(ctx, uint64(i), &buf)
if err != nil {
return err
}
if done {
fmt.Println("Checkpoint restore completed")
```

```
break
}
}
return nil
}
```

2. 청크 검증 및 복원

```
func restoreChunk(ctx context.Context, ndb db.NodeDB, chunk *ChunkMetadata, r i
hb := hash.NewBuilder()
tr := io.TeeReader(r, hb)
sr := snappy.NewReader(tr)
dec := cbor.NewDecoder(sr)
// 증명 재구성
var p syncer.Proof
p.V = v1ProofsVersion
for {
var entry []byte
if err := dec.Decode(&entry); err != nil {
if errors.Is(err, io.EOF) {
break
}
return fmt.Errorf("failed to decode chunk: %w", err)
```

```
}
p.Entries = append(p.Entries, entry)
}
p.UntrustedRoot = chunk.Root.Hash
// 청크 무결성 검증
chunkHash := hb.Build()
if !chunk.Digest.Equal(&chunkHash) {
return ErrChunkCorrupted
}
// 증명 검증
var pv syncer.ProofVerifier
ptr, err := pv.VerifyProof(ctx, chunk.Root.Hash, &p)
if err != nil {
return ErrChunkProofVerificationFailed
}
// 노드 데이터베이스에 임포트
return importChunkNodes(ndb, chunk, ptr)
}
```

자동 체크포인터

1. 체크포인터 설정

```go
type CheckpointerConfig struct {
Name  string  // 체크포인터 이름
Namespace common.Namespace // 네임스페이스
CheckInterval time.Duration // 체크 간격
RootsPerVersion  int  // 버전당 루트 수
Parameters *CreationParameters // 생성 매개변수
}

type CreationParameters struct {
Interval  uint64  // 체크포인트 간격 (라운드)
NumKept  uint64  // 유지할 체크포인트 수
ChunkSize  uint64  // 청크 크기
InitialVersion  uint64  // 초기 버전
ChunkerThreads  uint16  // 청킹 스레드 수
}
```

2. 체크포인터 생성 및 운영

```go
func  setupCheckpointer(ndb db.NodeDB, creator Creator) (Checkpointer, error) {
cfg := CheckpointerConfig{
Name: "runtime-checkpointer",
Namespace: namespace,
CheckInterval: 30 * time.Second,
```

```
RootsPerVersion: 2, // State + IO roots
Parameters: &CreationParameters{
Interval: 100, // 100 라운드마다
NumKept: 10, // 10개 체크포인트 유지
ChunkSize: 16 * 1024 * 1024, // 16MB 청크
InitialVersion: 0,
ChunkerThreads: 4, // 4개 스레드
},
}
checkpointer, err := NewCheckpointer(
context.Background(),
ndb,
creator,
cfg,
)
if err != nil {
return nil, err
}
return checkpointer, nil
}
```

3. 체크포인터 이벤트 처리

```go
func handleCheckpointerEvents(checkpointer Checkpointer) {
// 새 버전 알림
checkpointer.NotifyNewVersion(latestVersion)
// 강제 체크포인트
checkpointer.ForceCheckpoint(specificVersion)
// 체크포인트 완료 이벤트 감시
ch, sub, err := checkpointer.WatchCheckpoints()
if err != nil {
return
}
defer sub.Close()
go func() {
for version := range ch {
fmt.Printf("Checkpoint created for version %d\n", version)
}
}()
}
```

빠른 동기화

1. 체크포인트 기반 동기화

```go
func fastSync(nodeDB api.NodeDB, checkpointProvider ChunkProvider, targetRoot n
```

```go
// 체크포인트 메타데이터 가져오기
checkpoints, err := checkpointProvider.GetCheckpoints(context.Background(), &Get
Version: v1,
Namespace: targetRoot.Namespace,
RootVersion: &targetRoot.Version,
})
if err != nil {
return err
}
if len(checkpoints) == 0 {
return fmt.Errorf("no checkpoints available for version %d", targetRoot.Version
}
// 가장 적합한 체크포인트 선택
checkpoint := selectBestCheckpoint(checkpoints, targetRoot)
// 멀티파트 복원 시작
if err := nodeDB.StartMultipartInsert(targetRoot.Version); err != nil {
return err
}
defer nodeDB.AbortMultipartInsert()
// 복원기 생성
restorer, err := NewRestorer(nodeDB)
if err != nil {
return err
```

```
}
// 체크포인트 복원
if err := restoreFromCheckpoint(restorer, checkpointProvider, checkpoint); err
return err
}
// 복원 완료
return nodeDB.Finalize([]node.Root{targetRoot})
}
```

2. 점진적 동기화

```
func incrementalSync(nodeDB api.NodeDB, fromVersion, toVersion uint64) error {
for version := fromVersion + 1; version <= toVersion; version++ {
// 버전별 차이점 가져오기
writeLog, err := getWriteLogForVersion(nodeDB, version)
if err != nil {
return err
}
// 차이점 적용
if err := applyWriteLog(nodeDB, writeLog, version); err != nil {
return err
}
```

```go
fmt.Printf("Synced to version %d\n", version)
}
return nil
}
```

성능 최적화

1. 청킹 최적화

청크 크기 조정

```go
func optimizeChunkSize(networkBandwidth, latency time.Duration) uint64 {
// 네트워크 조건에 따른 최적 청크 크기 계산
baseSize := uint64(16 * 1024 * 1024) // 16MB 기본값
if latency > 100*time.Millisecond {
// 높은 지연시간: 큰 청크 사용
return baseSize * 2
}
if networkBandwidth < 10*time.Millisecond {
// 낮은 대역폭: 작은 청크 사용
return baseSize / 2
}
return baseSize
}
```

병렬 처리 최적화

```
func optimizeChunkerThreads(cpuCores int, ioCapacity int) uint16 {
// CPU 코어와 I/O 용량에 따른 최적 스레드 수
threads := uint16(cpuCores)
if ioCapacity < cpuCores {
threads = uint16(ioCapacity)
}
// 최소 2개, 최대 16개 스레드
if threads < 2 {
threads = 2
}
if threads > 16 {
threads = 16
}
return threads
}
```

2. 캐싱 전략

```
type checkpointCache struct {
cache map[string]*Metadata
mutex sync.RWMutex
}
```

```go
func (cc *checkpointCache) get(root node.Root) *Metadata {
cc.mutex.RLock()
defer cc.mutex.RUnlock()
key := fmt.Sprintf("%s-%d-%s", root.Namespace, root.Version, root.Hash)
return cc.cache[key]
}

func (cc *checkpointCache) put(root node.Root, metadata *Metadata) {
cc.mutex.Lock()
defer cc.mutex.Unlock()
key := fmt.Sprintf("%s-%d-%s", root.Namespace, root.Version, root.Hash)
cc.cache[key] = metadata
}
```

모니터링 및 진단

1. 체크포인트 메트릭

```go
type CheckpointMetrics struct {
CreationTime time.Duration // 생성 시간
ChunkCount int // 청크 수
TotalSize int64 // 전체 크기
```

```
CompressionRatio  float64  // 압축률

ThreadsUsed  int  // 사용된 스레드 수

}


func  collectCheckpointMetrics(metadata *Metadata, creationTime time.Duration) C

return CheckpointMetrics{

CreationTime: creationTime,

ChunkCount: len(metadata.Chunks),

TotalSize: calculateTotalSize(metadata),

CompressionRatio: calculateCompressionRatio(metadata),

}

}
```

2. 복원 진행률 추적

```
type RestoreProgress struct {

TotalChunks  int

CompletedChunks  int

BytesRestored  int64

StartTime time.Time

}
```

```go
func (rp *RestoreProgress) GetProgress() float64 {
if rp.TotalChunks == 0 {
return 0
}
return float64(rp.CompletedChunks) / float64(rp.TotalChunks)
}

func (rp *RestoreProgress) GetETA() time.Duration {
elapsed := time.Since(rp.StartTime)
progress := rp.GetProgress()
if progress == 0 {
return 0
}
totalTime := time.Duration(float64(elapsed) / progress)
return totalTime - elapsed
}
```

3. 오류 처리 및 복구

```go
func handleCheckpointErrors(err error) error {
switch {
case errors.Is(err, ErrChunkCorrupted):
// 손상된 청크: 다시 다운로드
```

```
return retryChunkDownload()
case errors.Is(err, ErrChunkProofVerificationFailed):
// 증명 검증 실패: 체크포인트 중단
return abortCheckpointRestore()
case errors.Is(err, ErrRestoreAlreadyInProgress):
// 이미 복원 중: 기존 복원 상태 확인
return checkExistingRestore()
default:
return err
}
}
```

사용 예제

완전한 체크포인트 워크플로우

```
func completeCheckpointWorkflow() error {
// 1. 노드 데이터베이스 설정
nodeDB, err := setupNodeDatabase()
if err != nil {
return err
}
defer nodeDB.Close()
```

```go
// 2. 체크포인트 생성기 설정
creator, err := setupCheckpointCreator(nodeDB)
if err != nil {
return err
}
// 3. 체크포인터 설정
checkpointer, err := setupCheckpointer(nodeDB, creator)
if err != nil {
return err
}
// 4. 새 버전 알림
checkpointer.NotifyNewVersion(latestVersion)
// 5. 체크포인트 생성 대기
waitForCheckpointCreation(checkpointer)
// 6. 체크포인트 기반 빠른 동기화
return performFastSync(nodeDB, creator, targetRoot)
}
```

체크포인트 시스템은 Oasis Core의 확장성과 성능을 크게 향상시키는 핵심 기능으로, 대용량 상태 데이터의 효율적인 동기화를 가능하게 합니다.